

Regresion no parametrica

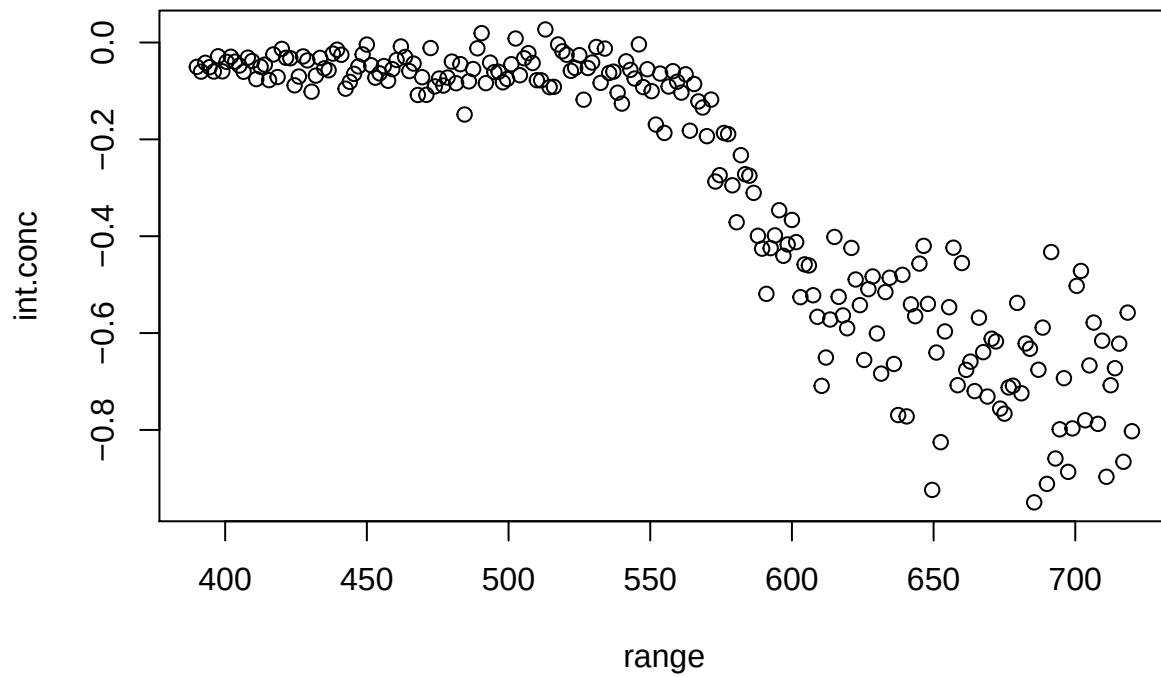
Ejercicio I

La tecnica conocida como LIDAR (light detection and ranging) usa la reflexion de luz de laser emitida para detectar compuestos quimicos en la atmosfera. Esta tecnica ha probado ser una herramienta muy eficiente para el monitoreo de la distribucion de diversos elementos polulantes en la atmosfera (Sigrist, 1994). En el archivo lidar.txt se encuentran datos medidos con a la tecnica LIDAR. La variable range es la distancia recorrida antes de que la luz sea reflejada de regreso hacia su fuente. La variable logratio es el logaritmo del cociente de la luz recibida de dos fuentes de luz laser de distinta frecuencia.

1. A partir de los datos de lidar.txt realice un diagrama de dispersion o scatter plot de range (eje x) vs. logratio (eje y). Describa la relacion entre ambas variables.

```
datos<-read.csv("lidar.txt", header =TRUE, sep="")
attach(datos)

plot(x=range,y=int.conc)
```



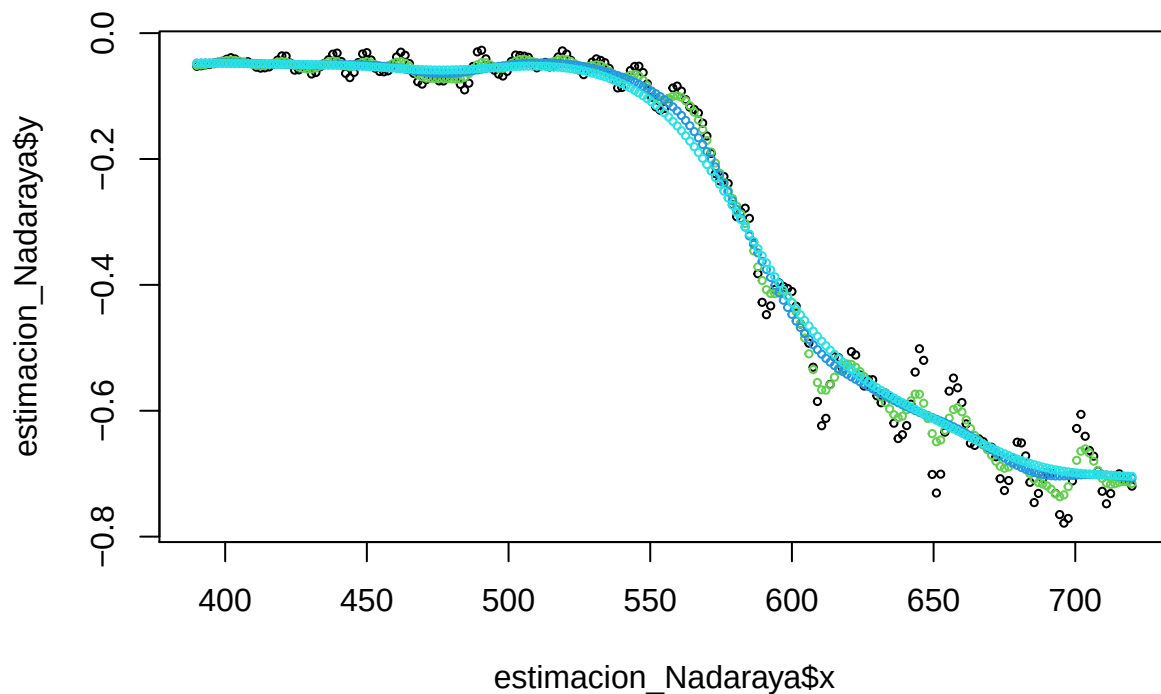
2. La funcion de R ksmooth computa el estimador de Nadaraya-Watson a partir de un conjunto de

datos $(x_1, y_1), \dots, (x_n, y_n)$ y lo evalúa en un conjunto de puntos intermedios. Mediante la función de R `ksmooth` estime la función de regresión r que relaciona a las variables `range` y `logratio`, tomando como variable explicativa a `range`, a partir de los datos dados usando el núcleo normal con ventana $h = 5$. Con la función de regresión estimada, obtenga estimaciones de la función de regresión en los valores observados de `range`. Grafique la función de regresión estimada. Repita para valores de la ventana $h = 10, 30, 50$ y superponga en el mismo plot los puntos correspondientes a las observaciones y el valor estimado de la función de regresión obtenida para $h = 5, 10, 30, 50$. Compare los resultados obtenidos con las 4 ventanas.

```

ventanas<-c(5,10,30,50)
for (i in 1:length(ventanas)) {
  estimacion_Nadaraya<-ksmooth(x=range, y=int.conc, kernel = "normal", bandwidth = ventanas[i])
  if(i==1){
    plot(x=estimacion_Nadaraya$x,y=estimacion_Nadaraya$y, cex=0.5)
  }
  else{points(x=estimacion_Nadaraya$x,y=estimacion_Nadaraya$y, col=i+1, cex=0.5)}
}

```



3. Para cada una de las 4 estimaciones obtenidas en el ítem anterior compute el Error Cuadrático de Predicción Promediado (ECP $P(h)$). ¿Cuál de las 4 ventanas consideradas da el menor ECP $P(h)$? ¿Cómo se puede justificar lo que está ocurriendo?

```

ventanas<-c(5,10,30,50)
ecpp<-c()
for (i in 1:length(ventanas)) {

```

```

estimacion_Nadaraya<-ksmooth(x=range, y=int.conc, kernel = "normal", bandwidth = ventanas[i])

valores_error <-c()
for (j in 1:length(datos)) {
  y_estimado<-approxfun(estimacion_Nadaraya)(range[j])
  valores_error <- c(valores_error,(int.conc[j]-y_estimado)**2)
}
ecpp<-c(ecpp,sum(valores_error)/length(datos))
}
ecpp

```

```
## [1] 3.658517e-05 4.513138e-05 8.151639e-05 7.677839e-05
```

4. Halle mediante el criterio de Convalidacion Cruzada CV (h) la ventana optima. Realice la busqueda en una grilla para valores de h entre 3 y 165 con paso 1. Realice un plot de h vs. CV (h).

```

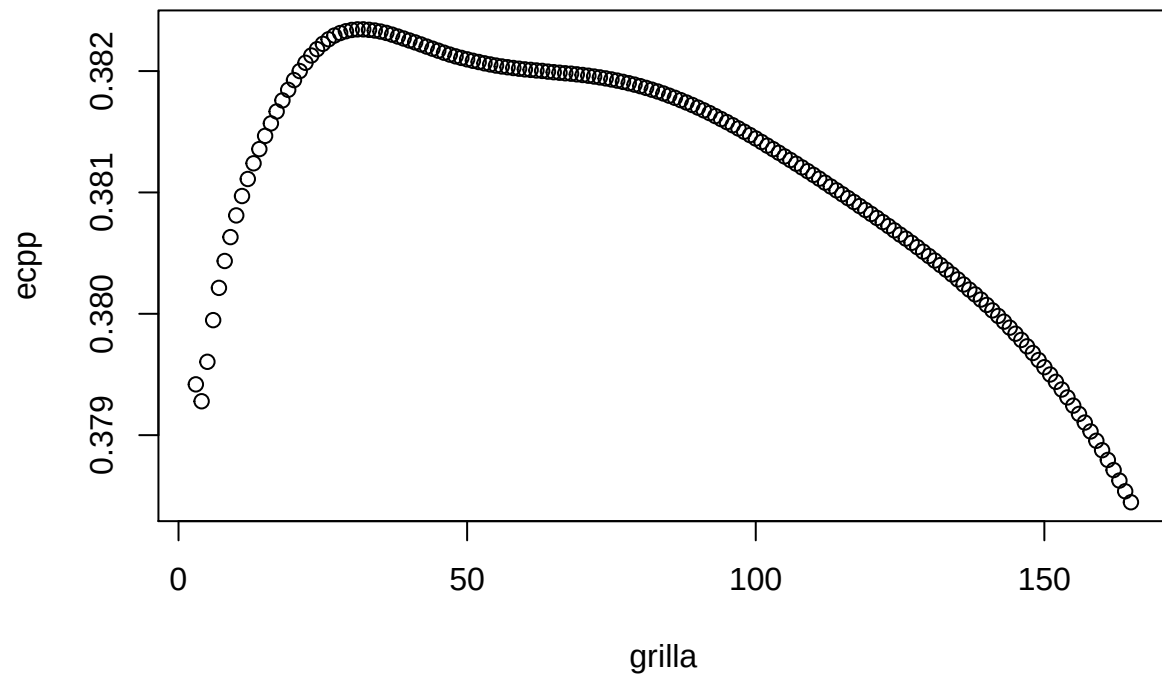
grilla<-seq(3,165,by=1)
num_random<-sample(1:length(range),length(range)*0.2)
datos_entrenamiento<-datos[-num_random,]
datos_testeo<-datos[num_random,]

ecpp<-c()
for (i in 1:length(grilla)) {
  estimacion_Nadaraya<-ksmooth(x=datos_entrenamiento$range, y=datos_entrenamiento$int.conc, kernel = "normal", bandwidth = grilla[i])

  valores_error <-c()
  for (j in 1:length(datos_testeo)) {
    y_estimado<-approxfun(estimacion_Nadaraya)(range[j])
    valores_error <- c(valores_error,((datos_testeo$int.conc[j]-y_estimado)**2))
  }
  ecpp<-c(ecpp,sum(valores_error)/length(datos_testeo))
}

plot(grilla,ecpp)

```

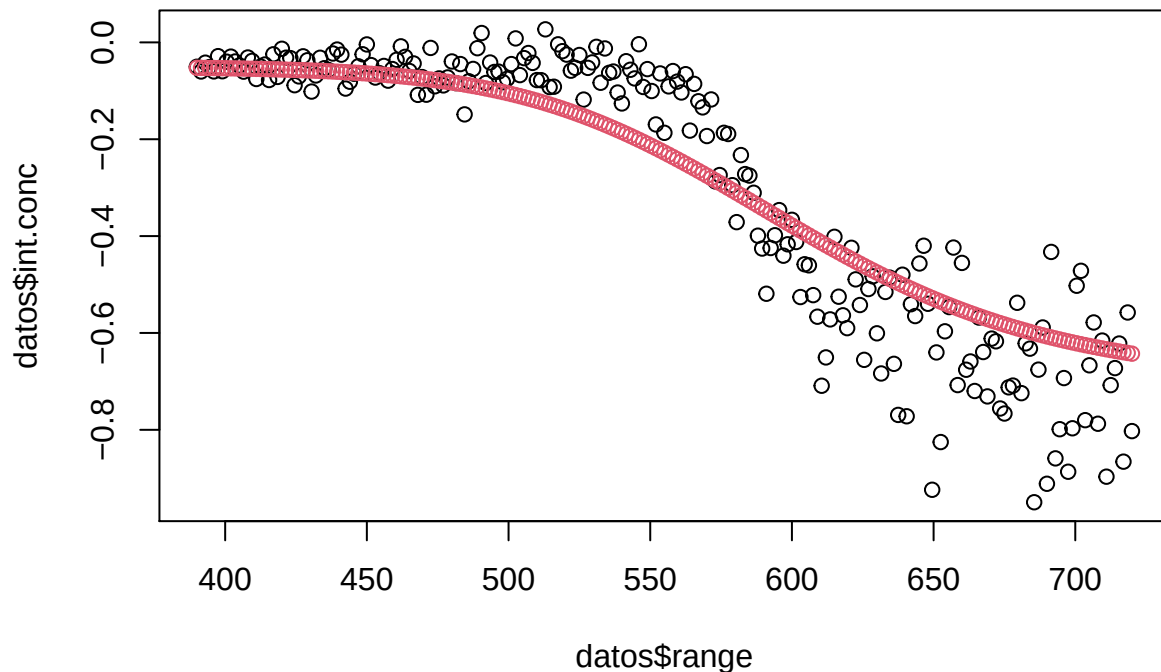


```
h_optima<-grilla[which.min(ecpp)]
h_optima
```

```
## [1] 165
```

5. Grafique los puntos observados y la funcion de regresion estimada por el metodo de Nadaraya-Watson con la ventana optima hallada.

```
estimacion_Nadaraya<-ksmooth(x=range, y=int.conc, kernel = "normal", bandwidth = h_optima)
plot(datos$range,y=datos$int.conc)
points(x=estimacion_Nadaraya$x,y=estimacion_Nadaraya$y, col=2)
```



6. Compute el Error Cuadratico de Prediccion Promediado de la estimacion provista por el metodo de Nadaraya-Watson con la ventana optima hallada: ECPP (hopt)

```
# hoptima del punto 4
estimacion_Nadaraya<-ksmooth(x=range, y=int.conc, kernel = "normal", bandwidth = h_optima)

valores_error <-c()
for (j in 1:length(datos)) {
  y_estimado<-approxfun(estimacion_Nadaraya)(range[j])
  valores_error <- c(valores_error,(int.conc[j]-y_estimado)**2)
}
```

7. Compute la Perdida de Convalidacion Cruzada asociada al la estimacion provista por el metodo de Nadaraya-Watson con la ventana optima hallada. CV (hopt)

```
num_random<-sample(1:length(range),length(range)*0.2)
datos_entrenamiento<-datos[-num_random,]
datos_testeo<-datos[num_random,]

estimacion_Nadaraya<-ksmooth(x=datos_entrenamiento$range, y=datos_entrenamiento$int.conc, kernel = "normal", bandwidth = h_optima)

valores_error <-c()
for (j in 1:length(datos_testeo)) {
  y_estimado<-approxfun(estimacion_Nadaraya)(range[j])
  valores_error <- c(valores_error,((datos_testeo$int.conc[j]-y_estimado)**2))
}
```

```
ecpp <- sum(valores_error)/length(datos_testeo)
ecpp
```

```
## [1] 0.07216982
```

8.

8. ¿Que se puede hacer con knn?

Ejercicio II. Bonus Track Revisitemos la guía de alturas con este nuevo marco teórico; elija h y k con las diferentes propuestas consideradas en clase.